

Je travaille sur une évaluation d'interopérabilité en santé. Le sujet que j'ai choisi est **« Admission d'un patient »**.

Je dois concevoir un prototype d'application capable de récupérer les informations d'un patient depuis un serveur **HL7 FHIR réel**, puis de les utiliser pour construire une représentation au format **HL7 v2** correspondant à une admission.

Le serveur FHIR qui m'est fourni est :

<https://hapi.fhir.org/baseR4>

Je veux travailler avec ce serveur réel et ne pas remplacer ses données par des données fictives.

Tu es mon assistant spécialisé en **HL7 FHIR, HL7 v2 et interopérabilité des systèmes d'information de santé**.

Très important

Nous sommes actuellement dans la **phase de conception**.

Je ne veux **aucun code pour le moment**.

Mon objectif est d'abord de construire un **cahier des charges clair, réaliste et suffisamment précis** pour qu'ensuite je puisse demander à une IA de développement de réaliser le prototype sans qu'elle ait à prendre elle-même des décisions importantes.

Je veux aussi que tu adoptes une démarche de réflexion critique : si tu proposes quelque chose, explique-moi pourquoi c'est pertinent pour notre cas. Si une information n'est pas certaine, ne l'invente pas et indique clairement **« À VÉRIFIER »**.

1. Commencer par comprendre le besoin

Commence par reformuler avec tes propres mots ce que tu comprends du sujet.

Je veux que tu m'expliques simplement :

- * quel est le besoin métier ;
- * quel problème l'application doit résoudre ;
- * quel est son objectif principal ;
- * qui va l'utiliser ;
- * comment se déroule concrètement une admission ;
- * quelles informations sont nécessaires ;
- * quel résultat l'utilisateur doit obtenir ;
- * ce qui fait partie du projet ;
- * ce qui ne fait pas partie du projet.

Propose ensuite un scénario concret, depuis la recherche du patient jusqu'à la confirmation de son admission.

Je veux que le scénario soit réaliste mais reste adapté à un prototype d'évaluation.

2. Identifier les ressources FHIR réellement utiles

À partir du scénario, détermine quelles ressources FHIR sont réellement nécessaires.

Pour chaque ressource proposée, explique :

- * à quoi elle sert dans notre scénario ;
- * quelles informations nous allons réellement utiliser ;
- * comment elle est liée aux autres ressources ;
- * pourquoi elle est nécessaire.

Porte une attention particulière à :

- * `Patient` pour l'identité ;
- * `Encounter` pour l'admission/séjour ;
- * `Organization` si l'établissement est nécessaire ;
- * `Practitioner` si un professionnel doit être identifié.

Ne choisis pas une ressource simplement parce qu'elle existe dans FHIR. Je veux comprendre son utilité dans **notre cas d'usage**.

3. Vérifier la version FHIR et le serveur

Le serveur fourni est :

`https://hapi.fhir.org/baseR4`

Vérifie ce que nous pouvons réellement savoir sur ce serveur avant de faire des hypothèses.

Je veux notamment identifier :

- * la version FHIR ;
- * le `CapabilityStatement` ;
- * les ressources disponibles ;
- * les opérations supportées ;
- * les paramètres de recherche disponibles ;
- * les profils éventuellement présents ;
- * les éventuels Implementation Guides ;

* les contraintes importantes.

Si une information ne peut pas être confirmée, écris **« À VÉRIFIER »**.

Ne transforme jamais une hypothèse en certitude.

4. Définir les échanges avec le serveur FHIR

Décris ensuite comment notre application communiquera avec le serveur.

Pour chaque échange, donne :

- * la méthode HTTP ;
- * l'endpoint ;
- * la ressource concernée ;
- * le but de la requête ;
- * les paramètres utilisés ;
- * le type de réponse attendu ;
- * les erreurs possibles.

Par exemple, nous pouvons avoir besoin de :

- * vérifier le serveur ;
- * rechercher un patient ;
- * récupérer un patient ;
- * rechercher ou récupérer un Encounter ;
- * récupérer éventuellement une Organization ou un Practitioner.

Mais ne suppose pas qu'un endpoint ou un paramètre est disponible sans le vérifier.

Le prototype doit utiliser les **données réellement présentes sur le serveur HAPI FHIR**.

5. Déterminer le message HL7 v2 pour l'admission

Le format cible imposé par le sujet est **HL7 v2**.

Pour une admission, vérifie si le message **ADT^A01** est bien le message adapté à notre scénario.

Si c'est le cas, explique simplement pourquoi.

Présente ensuite les principaux segments nécessaires, par exemple :

* `MSH`

- * `EVN`
- * `PID`
- * `PV1`

Pour chacun, explique :

- * son rôle ;
- * les informations qu'il contient ;
- * quelles données FHIR peuvent servir à le construire.

Si certaines informations nécessaires au message HL7 v2 ne sont pas disponibles dans FHIR, signale-le clairement.

6. Construire le mapping FHIR → HL7 v2

Je veux ensuite une première table de mapping.

Utilise cette structure :

Ressource FHIR	Champ FHIR	Signification	Segment/champ HL7 v2	Transformation	Risque ou perte
----------------	------------	---------------	----------------------	----------------	-----------------

Analyse notamment :

- * identifiant patient ;
- * nom ;
- * prénom ;
- * date de naissance ;
- * sexe ;
- * informations d'admission ;
- * date et heure ;
- * établissement ;
- * service/unité ;
- * professionnel si nécessaire.

Pour chaque correspondance, indique si elle est :

- * exacte ;
- * transformée ;
- * approximative ;
- * ou impossible à déterminer.

Si une correspondance n'est pas suffisamment fiable, écris :

****« Correspondance non déterminée »****

Je ne veux surtout pas de mapping inventé uniquement pour remplir le tableau.

7. Vérifier l'interopérabilité sémantique

Je veux que tu m'aides à aller au-delà de la simple communication technique.

Analyse les terminologies et les systèmes de codification qui peuvent intervenir dans notre scénario :

- * identifiants ;
- * établissement ;
- * service ;
- * rôle ;
- * sexe ;
- * classe d'Encounter ;
- * autres codes nécessaires.

Pour chaque élément important, explique :

1. à quoi il sert ;
2. quel système de codification est utilisé ;
3. où il apparaît dans FHIR ;
4. où il apparaît dans HL7 v2 ;
5. comment la transformation peut être réalisée.

Fais bien la différence entre :

****syntaxe → structure → sémantique****

Je veux notamment montrer dans mon évaluation qu'une API qui fonctionne avec HTTP ne garantit pas forcément une bonne interopérabilité.

8. Décrire l'architecture avec le ReEIF

Présente l'architecture de notre solution en utilisant les six couches du ReEIF étudiées en cours :

1. Juridique
2. Politique / gouvernance
3. Processus de soin
4. Information
5. Applications
6. Infrastructure

Pour chaque couche, explique ce qui concerne notre prototype, les contraintes éventuelles et ce qui devra être vérifié.

Je veux que l'architecture reste réaliste pour un prototype d'évaluation et qu'elle ne soit pas inutilement complexe.

9. Définir les contraintes du futur développement

À partir de tout ce que nous avons défini, établis les règles que le futur développeur devra respecter.

Le prototype devra notamment :

- * communiquer avec le serveur FHIR réel ;
- * respecter sa version FHIR ;
- * utiliser les ressources réellement disponibles ;
- * ne pas utiliser de données fictives à la place des données du serveur ;
- * gérer les erreurs HTTP ;
- * gérer les réponses FHIR inattendues ;
- * afficher les données réellement récupérées ;
- * produire une représentation HL7 v2 d'admission ;
- * permettre de vérifier le mapping ;
- * garder une trace minimale des échanges ;
- * fonctionner avec plusieurs patients ;
- * ne pas dépendre de valeurs codées en dur.

10. Définir des critères d'acceptation

Je veux maintenant transformer le besoin en critères que je pourrai réellement vérifier pendant l'évaluation.

Utilise une formulation du type :

****Étant donné que...**
Lorsque...
Alors...**

Par exemple :

****AC1 — Recherche d'un patient****

Étant donné que le serveur FHIR est accessible, lorsque l'utilisateur recherche un patient existant, alors l'application doit retourner les données du patient réellement présentes sur le serveur.

Propose une liste complète de critères d'acceptation.

Pour chaque critère, explique également comment je peux le tester.

11. Préparer le plan de tests

Avant de coder, définis une stratégie de test.

Les tests doivent notamment couvrir :

- * connexion au serveur ;
- * récupération de la version FHIR ;
- * recherche d'un patient ;
- * patient existant ;
- * patient inexistant ;
- * récupération d'un Encounter ;
- * données obligatoires absentes ;
- * données incorrectes ;
- * erreur HTTP ;
- * réponse FHIR inattendue ;
- * génération du message ADT ;
- * vérification du mapping ;
- * vérification des terminologies ;
- * cohérence entre les données reçues et les données affichées.

Présente les tests sous la forme :

ID	Objectif	Entrée	Action	Résultat attendu	Critère de réussite
----	----------	--------	--------	------------------	---------------------

Je veux des tests réellement réalisables avec le serveur fourni.

12. Prévoir la vérification de ce que propose l'IA

Je vais utiliser une IA pour m'aider à développer le prototype.

Je veux donc que tu identifies clairement ce que je devrai contrôler avant d'accepter une proposition de l'IA.

Pour chaque point important, indique s'il dépend :

- * de la spécification FHIR ;
- * de la version FHIR ;
- * du serveur HAPI ;

- * d'un profil ;
- * d'un Implementation Guide ;
- * de la spécification HL7 v2 ;
- * d'une terminologie ;
- * ou simplement d'une hypothèse.

Je veux notamment éviter les erreurs classiques :

- * mauvaise ressource ;
- * mauvais champ ;
- * mauvais endpoint ;
- * mauvaise version ;
- * mauvais message HL7 ;
- * mauvais mapping ;
- * mauvaise terminologie ;
- * confusion entre syntaxe et sémantique ;
- * utilisation d'une fonctionnalité que le serveur ne supporte pas.

13. Prévoir une vraie boucle de développement

Je veux que le futur développement suive cette logique :

****CONCEVOIR**
 → DEMANDER À L'IA
 → VÉRIFIER
 → TESTER SUR LE SERVEUR RÉEL
 → IDENTIFIER LES ERREURS
 → CORRIGER
 → RETESTER
 → VALIDER******

Explique comment appliquer cette méthode concrètement.

L'idée est simple : une modification proposée par l'IA n'est pas considérée comme correcte simplement parce que le code fonctionne.

Elle doit être vérifiée par rapport aux spécifications, testée sur le serveur et validée.

14. Identifier les limites et pertes d'information

Identifie les informations qui pourraient :

- * être absentes du serveur ;
- * être représentées différemment en FHIR et HL7 v2 ;

- * être difficiles à mapper ;
- * être perdues lors de la transformation ;
- * être ambiguës au niveau sémantique.

Pour chaque cas, explique l'impact possible sur l'interopérabilité.

15. Construire le cahier des charges final

À partir de toute l'analyse précédente, construis un cahier des charges propre et cohérent avec les sections suivantes :

1. Contexte
2. Besoin métier
3. Objectifs
4. Utilisateurs
5. Scénario d'utilisation
6. Périmètre
7. Ressources FHIR
8. Version et spécifications
9. Interactions avec le serveur FHIR
10. Format cible HL7 v2
11. Mapping FHIR → HL7 v2
12. Terminologies
13. Architecture
14. Analyse ReEIF
15. Contraintes
16. Gestion des erreurs
17. Traçabilité
18. Critères d'acceptation
19. Plan de tests
20. Risques et points à vérifier
21. Limites et pertes d'information
22. Stratégie de validation et d'itération

16. Faire une auto-vérification avant de terminer

Avant de me donner la version finale, fais une dernière vérification.

Vérifie notamment :

- * Est-ce que les ressources FHIR choisies répondent vraiment au besoin ?
- * Est-ce que le scénario métier est cohérent ?
- * Est-ce que les endpoints correspondent réellement au serveur ?
- * Est-ce que la version FHIR est correcte ?

- * Est-ce que le message HL7 v2 choisi correspond bien à une admission ?
- * Est-ce que le mapping FHIR → HL7 v2 est justifié ?
- * Est-ce que les terminologies sont cohérentes ?
- * Est-ce que les critères d'acceptation sont testables ?
- * Est-ce que chaque fonctionnalité possède au moins un test ?
- * Est-ce qu'il reste des hypothèses ?
- * Quelles informations devront encore être vérifiées directement sur le serveur ?

Si tu trouves une erreur dans ta première analyse, corrige-la.

Présente ensuite :

Version 1

Le cahier des charges initial.

Auto-vérification

Les problèmes ou incertitudes que tu as identifiés.

Version finale corrigée

Le cahier des charges après correction.

Règle générale

Je préfère une réponse qui dit ****« À VÉRIFIER »**** plutôt qu'une information inventée.

Je veux que le résultat montre une vraie compréhension de l'interopérabilité en santé et pas seulement une utilisation technique de FHIR.

****Ne génère aucun code.****

Le résultat final doit pouvoir servir directement de base à une prochaine étape où je demanderai à une IA de développement de construire le prototype.